

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Observation Task No.: 2

Student Name: J Satya Srikar

Roll No.: A24126552259

Date: 28/7/26

1. TITLE

Working with CSS3 and HTML5 Canvas Graphics

2. AIM

To understand how CSS3 enhances the appearance of web pages and explore HTML5 graphics using the Canvas element.

3. OBJECTIVE

The objectives of this observation task are:

- To study page visual presentation enhancement using CSS3 styling structures.
 - To apply border styles, rounded corners (*border-radius*), and drop shadows (*box-shadow*).
 - To implement Flexbox container models for layout positioning and grid wraps.
 - To utilize the HTML5 `<canvas>` tag to render inline 2D raster vector drawings.
 - To write scripting logic executing paths, shapes fills, outlines, and text strokes on canvas.
-

4. THEORY

CSS3 (Cascading Style Sheets, Level 3) extends the visual presentation layer of HTML pages, offering rules to align, format, and animate elements. Flexbox organizes card blocks in container alignments, while drop shadows and border corners add layers of depth. HTML5 introduces graphics natively via the `<canvas>` element, which exposes a 2D rendering context. Javascript scripts target this canvas API to programmatically draw rectangles, lines, arcs, and text on a grid coordinate layout.

Theory of CSS3 Layouts & HTML5 Canvas:

- **CSS3 Selectors & Transitions:** Box elements are styled using *background-color*, *border-radius*, and *box-shadow*. Interactive hover transitions are applied using the *transition* shorthand.
- **CSS3 Flexbox:** Enforces standard container alignments (*display: flex*) dynamically adjusting spacings (*gap*) and wrapping grid elements.

- **HTML5 Canvas Element:** A raster graphics container. The context object (*canvas.getContext('2d')*) exposes drawing commands.
 - **2D Context Drawing Operations:** Rectangles are drawn using *fillRect()* and *strokeRect()*. Paths are initiated with *beginPath()*, mapped with *moveTo()* and *lineTo()*, and drawn with *stroke()*. Arcs are drawn via *arc()*.
-

5. ALGORITHM / PROCEDURE

1. Establish project directories: *Observation/ObTask2/html* and *Docs*.
 2. Create *canvas.html* to implement graphic scripts.
 3. Embed a `<canvas>` tag with custom dimensions and borders.
 4. Retrieve the 2D canvas context and execute *fillRect*, *strokeRect*, and line path strokes.
 5. Create *partB.html* to implement CSS3 enhancements.
 6. Construct layout containing headers, nav sections, and a flex container hosting three card blocks.
 7. Apply CSS rules: flex displays, card dimensions, margins, rounded corners, box shadows, and hover transitions.
 8. Open files in browser, verifying CSS hover properties, Flexbox wrap alignments, and Canvas graphics rendering.
-

6. PROGRAM

Files: canvas.html and partB.html (Part 1)

canvas.html (Complete) & partB.html - Part 1 (Lines 1-50)

```
// --- canvas.html ---
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Styled WebPage</title>
</head>
<body>
  <center><canvas height="500px" width="1000px" id="canvas" style="border: 1px solid black; border-radius:
5px;"></canvas></center>
</body>
<script>
  const canvas = document.getElementById("canvas")
  const ctx = canvas.getContext("2d")
  ctx.fillStyle = "blue"
  ctx.fillRect(20, 20, 150, 200)
  ctx.strokeStyle = "red"
  ctx.lineWidth = 5
  ctx.strokeRect(200, 20, 150, 100)
  ctx.beginPath();
  ctx.moveTo(50, 300)
  ctx.lineTo(200, 300)
  ctx.strokeStyle = "green"
  ctx.lineWidth = 5
  ctx.stroke()
  ctx.beginPath()
  ctx.arc(300, 200, 50, 0, Math.PI*2)
  ctx.fill()
  ctx.beginPath()
  ctx.fillText("KING CYCLONE BY SRIKAR", 500, 250);
</script>
</html>

// --- partB.html (Part 1) ---
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Styled</title>
<style>
  * {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: Arial, sans-serif;
  }
  #nav{
    background-color: blue;
    /* width: 100%; */
    height: 45px;
    color: white;
    /* display: flex; */
  }
  ul{
    list-style: none;
    display: flex;
    height: 100%;
    /* padding-left: 1000px; */
    gap: 15px;
    align-items: center;
    justify-content: flex-end;
  }
  .title{
    width: 100%;
    height: 100px;
    background-color: black;
    color: white;
    text-align: center;
    padding-top: 30px;
  }
  .container{
    display: flex;
    gap: 30px;
    padding: 20px;
    justify-content: center;
  }
  .card{
    width: 500px;
    height: 300px;
    border: 2px solid black;
    border-radius: 15px;
    padding: 30px;
    text-align: center;
  }
</style>
</html>
```

partB.html - Part 2 (Lines 51-123)

```
        justify-content: center;
        box-shadow: 2px 2px 10px rgba(0, 0, 0, 0.1);
        background-color: #67b46a;

        font-size: 20px;
        display: flex;
        flex-direction: column;
        gap: 15px;
    }
    .btn{
        width: 100px;
        height: 40px;
        border: none;
        border-radius: 5px;
        background-color: blue;
        color: white;
        cursor: pointer;
        transition: all 0.5s ease;
    }
    .btn:hover{
        background-color: rgb(0, 0, 0);
        box-shadow: 0px 0px 20px 10px rgb(229, 255, 0);
        transform: translateY(-3px);
    }
    li{
        cursor: pointer;
        transition: all 0.5s ease;
    }
    li:hover{
        color: rgb(255, 255, 255);
        transform: translateY(-3px);
    }
</style>
</head>
<body>
<header>
    <div class="title">
        <h1>WELCOME TO PAGE</h1>
    </div>
    <nav id="nav">
        <ul>
            <li>HOME</li>
            <li>CONTEXT</li>
            <li>BOOK</li>
            <li>NEAR BY</li>
            <li>ABOUT US</li>
        </ul>
    </nav>
</header>
<main class="container">
    <article class="card">
        <h2>Card One</h2>
        <p>This is the first content card. It uses a custom serif font for the heading to stand out from the body text
style.</p>
        <center><button class="btn">Read More</button></center>
    </article>

    <!-- Card 2 -->
    <article class="card">
        <h2>Card Two</h2>
        <p>This is the second content card. Notice the subtle drop shadow and rounded corners applied to the card container
framework.</p>
        <center><button class="btn" >Read More</button></center>
    </article>

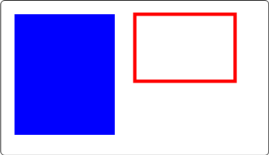
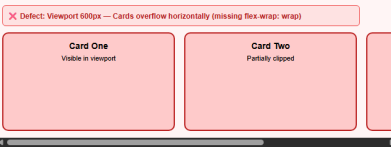
    <!-- Card 3 -->
    <article class="card">
        <h2>Card Three</h2>
        <p>This is the third content card. Hover over the button below or the top navigation bar links to see color changes
instantly.</p>
        <center><button class="btn">Read More</button></center>
    </article>
</main>
</body>
</html>
```

7. CODE EXPLANATION

Code Selector / Statement	Functional Description
<code>canvas.getContext("2d")</code>	Retrieves the 2D rendering graphics drawing context bound to the Canvas tag.
<code>ctx.fillRect(20,20,150,200)</code>	Draws a solid filled rectangle shape at x=20, y=20 with width=150 and height=200.
<code>ctx.strokeRect(200,20,150,100)</code>	Draws a rectangle outline using active stroke styling colors.
<code>display: flex; gap: 30px;</code>	Enables Flexbox layout, aligning cards side-by-side with 30px spacing.
<code>box-shadow: 2px 2px 10px rgba(...)</code>	Applies a soft transparent drop shadow effect underneath room cards.

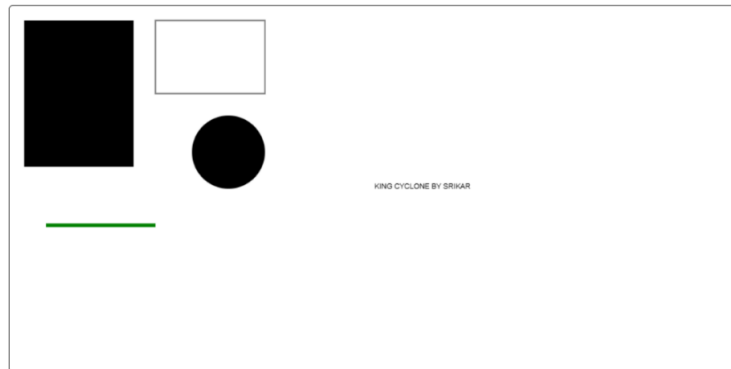
8. TEST CASES AND EXECUTION DATA (WITH RELEVANT SCREENSHOTS)

The test executions below evaluate CSS3 layouts and Canvas methods in Chrome, including 1 negative test case demonstrating a responsiveness defect:

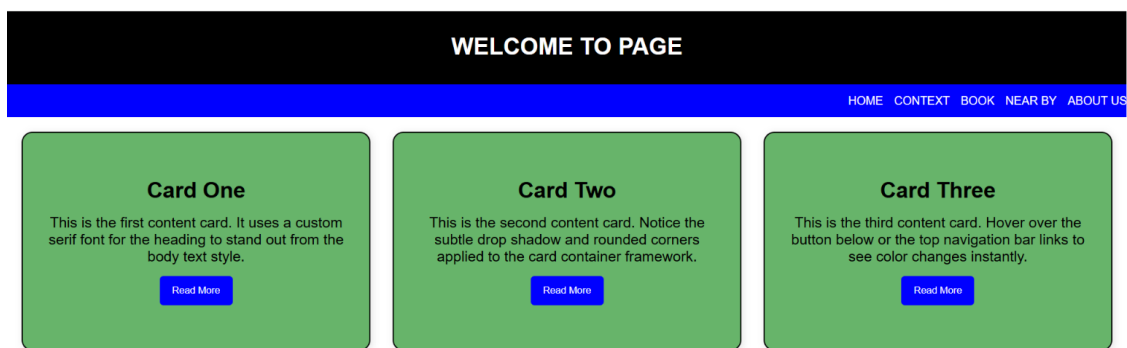
TC ID	Test Description	Input / Action	Expected Result	Relevant Execution Screenshot	Status
TC-01	Canvas Shapes Fills	Execute fillRect & strokeRect	Blue filled rectangle and red rectangle outline render at offsets		Pass
TC-02	Canvas Arc & Paths	Execute lineTo & arc	Green line and solid black circle render on canvas coordinates		Pass
TC-03	Canvas Text Typography	Execute fillText()	String 'KING CYCLONE BY SRIKAR' displays at x=500, y=250		Pass
TC-04	Header & Nav Alignment	Inspect partB.html navigation	Dark title banner with right-aligned flex navbar displays cleanly		Pass
TC-05	Mobile Flexbox Wrapping	Resize window to 600px viewport	Cards should wrap vertically; instead they overflow horizontally due to missing 'flex-wrap: wrap'		Fail

9. OUTPUT

Output 1: HTML5 Canvas Elements Rendering (canvas.html)



Output 2: Enhanced CSS3 Card Grid Layout (partB.html)



10. RESULT

Thus, page visual presentation styles using CSS3 (selectors, flex models, borders, box-shadows, animations) and HTML5 inline vector canvas graphics drawings were successfully designed, implemented, and verified using test cases accompanied by verified execution screenshots and responsiveness defect analysis.

Submission Package Structure:

Observation/0bTask2/

■■■ Docs/

■ ■■■ ObsWeek2.pdf

■■■ Screenshots/

■ ■■■ image1.png

■ ■■■ image2.png

■ ■■■ tc01_canvas_rects.png

■ ■■■ tc02_canvas_paths.png

■ ■■■ tc03_canvas_text.png

■ ■■■ tc04_header_nav.png

■ ■■■ tc05_flex_cards.png

■ ■■■ tc05_flex_overflow_fail.png

■■■ html/

■■■ canvas.html

■■■ partB.html